by

Zhuo Li

A Thesis Submitted in Partial Fulfillment
of the Requirements for the Degree of

Master of Science

Department of Computer Science
University of New Mexico

2026

Contents

1 Introduction **1**

2 Background **2**

2.1 State Estimation and Control Under Sensing Errors 2

2.2 Data-Driven Estimation from Auxiliary Measurements 3

2.3 PAC Modeling and Statistical Analysis 4

2.4 Runtime Assurance and the Simplex Architecture 5

2.5 GPU Power Modeling and Feedback Power Control 6

3 Preliminaries and Problem Formulation **8**

3.1 Notation 8

3.2 Reliable State Estimation 9

3.3 Scenario Optimization 10

3.4 Probably Approximately Correct Approximation 11

3.5 Problem Formulation 13

3.6 Framework Overview 14

3.6.1 Offline Construction and Evaluation 14

3.6.2 Online Estimation and Control Mode Selection 14

4 Piecewise PAC Approximation **17**

4.1 PAC Guarantees under Regional Resampling 18

4.1.1 Regional Sampling Mechanism 18

4.1.2 Local and Overall Guarantees 18

4.2 PAC Guarantees from a Fixed Dataset 18

4.2.1 Approximation on a Smaller Region 18

4.2.2 Approximation on Entire Regions 18

4.2.3 Approximation on Sample-Insufficient Regions 18

5 Adaptive Piecewise Polynomial Estimator Construction **19**
5.1 Polynomial Representation 19
5.1.1 Piecewise Linear Approximation 20
5.2 Adaptive Degree Selection 21
5.3 Impurity-Based Subdivision 23
5.3.1 Residuals and Impurity Criterion 24
5.3.2 Subdivision Procedure 24

6 Online Control and Mode Selection **27**
6.1 Online Decision Making 27
6.2 Safe Prediction Horizon and Conservative Control 29
6.2.1 Estimation of L_{safe} 29

7 Experiments **31**
7.1 GPU Power Control Case Study 31
7.1.1 Experiment Setup 31
7.1.2 Evaluation on Simulator I 35
7.1.3 Evaluation on Simulator II 36
7.1.4 Evaluation on the GPU Server 37

8 Conclusion **38**

List of Figures

3.6.1 Overview of the Simplex control framework. 15

List of Tables

7.1 Evaluation of PAC estimators on Simulator I. The results from the maximum-residual subdivision are shown in parentheses, while those outside parentheses use residual range subdivision. Legend - K: number of subdivision; $K_{\leq\epsilon_{\min}}$: number of subdivisions $\epsilon \leq \epsilon_{\min}$; $\epsilon_{\text{lo}}/\epsilon_{\text{hi}}$: smallest/largest fitted error bounds among these subdivisions; Time: total construction time of the estimator; L_{safe}: evaluated safe prediction horizon.**Zhuo** fill in the data 35

7.2 Evaluation of PAC estimators on Simulator II. The results from the maximum-residual subdivision are shown in parentheses, while those outside parentheses use residual range subdivision. Legend - K: number of subdivision; $K_{\leq\epsilon_{\min}}$: number of subdivisions $\epsilon \leq \epsilon_{\min}$; $\epsilon_{\text{lo}}/\epsilon_{\text{hi}}$: smallest/largest fitted error bounds among these subdivisions; Time: total construction time of the estimator; L_{safe}: evaluated safe prediction horizon.**Zhuo** fill in the data 36

Chapter 1

Introduction

Chapter 2

Background

This chapter reviews research related to data-driven state estimation and statistical guarantees for learned models. It begins with state estimation, attack detection, and recovery control under sensing errors, followed by estimation from auxiliary measurements. PAC modeling and statistical analysis are then reviewed as approaches to characterizing reliability from finite samples. The chapter also introduces runtime assurance through the Simplex architecture and reviews GPU power modeling and control as the application background.

2.1 State Estimation and Control Under Sensing Errors

Unreliable measurements affect both state estimation and feedback control. Research on this problem includes reconstructing the state from corrupted sensor data, detecting inconsistencies in measurements, and recovering the system after an attack. These tasks address different stages of the response to sensing errors. Estimation supplies information about the system state, detection identifies potentially corrupted measurements, and recovery control determines the actions required to restore safe operation.

Classical filtering estimates the state by combining a dynamical model with measurements. Kalman [1] developed a recursive linear estimator that updates a model-based prediction using new measurements and propagates the estimation-error covariance. The resulting uncertainty characterization depends on the assumed dynamics and noise statistics. Sensor faults such as persistent bias or stuck readings may require additional fault models or changes to the estimation procedure when they are not represented by those assumptions.

Pajic et al. [2] studied attack-resilient state estimation with bounded noise and modeling errors. Their approach reconstructs the state despite corruption of a subset of the sensors and characterizes the resulting worst-case estimation error. The guarantees depend on the system model, sensor redundancy, and assumptions about the attacks. The work also examines implementation issues

through a resilient cruise-control application.

Sensor redundancy can also be used to estimate a physical quantity by combining measurements from several sensors. Ivanov et al. [3] developed an attack-resilient sensor fusion method for sensors measuring the same physical variable. The method accounts for the specified precision of each sensor and tolerates corruption of a subset of the sensors. The work also examines how communication schedules on a shared bus affect the influence of an attacker on the fused estimate.

Alongside reconstructing reliable values from sensor data, detection methods check whether measurements are consistent with expected physical behavior. Choi et al. [4] derived and monitored control invariants to detect physical attacks against robotic vehicles. These invariants describe relationships associated with the vehicle's control behavior and allow deviations to be checked during execution. Quinonez et al. [5] developed SAVIOR, which uses robust physical invariants to assess sensor information in autonomous vehicles. Such methods use physical consistency to identify suspicious measurements. After detection, a separate recovery procedure is needed to determine how the system should respond.

Zhang et al. [6] studied real-time attack recovery using linear approximations. Their method estimates the current state, reasons about reachable states, and computes a recovery control sequence. The objective is to steer the system toward a target while avoiding unsafe behavior under a sensor attack.

2.2 Data-Driven Estimation from Auxiliary Measurements

The information needed for monitoring and control can also be obtained by learning relationships between measured variables. Data-driven estimation uses these relationships to infer quantities that are difficult to measure directly. In the process industry, such estimators are commonly called soft sensors. Kadlec et al. [7] reviewed data-driven soft sensors for predicting process variables, monitoring processes, and detecting faults. Their review covers modeling techniques, industrial applications, and the characteristics of process data that affect model development. A soft sensor can provide an estimate of a required quantity from other measured variables without requiring a complete physical description of the process.

The development of a soft sensor involves selecting informative inputs, preparing the data, and constructing a predictive model. Measurement noise and missing values affect the data available for fitting, while changes in operating conditions can alter the relationship between the inputs and the target quantity [7].

Data-driven estimation has also been incorporated into attack recovery. Zhang et al. [8] developed a data-predictive recovery method for nonlinear cyber-physical systems. Their formulation assumes that compromised sensors can be identified and uses uncorrupted sensors to reduce the

accumulation of uncertainty. Under its stated conditions, the recovery procedure avoids unsafe states, reaches a target set within a conservative deadline, and remains within that set for a specified period. The estimates therefore support a recovery objective with explicit safety and timing requirements.

The reliability of a data-driven estimator depends on both its approximation error and the sampling conditions under which it is constructed. This thesis examines regional resampling as a reference setting for studying PAC guarantees, while focusing on estimator construction from a fixed dataset when additional samples are unavailable. The objective is to characterize the probability that the estimation error exceeds a specified bound under the input distribution. PAC learning and scenario optimization provide a basis for studying such guarantees.

2.3 PAC Modeling and Statistical Analysis

The Probably Approximately Correct (PAC) framework studies the sample requirements for learning a model that satisfies a specified accuracy criterion with high confidence. Haussler [9] surveyed the framework and its role in computational learning theory. Formal definitions and sample requirements are introduced in the next chapter.

Scenario optimization connects finite sampled constraints with probabilistic guarantees. Calafiore and Campi [10] studied robust design problems in which uncertain constraints are replaced by constraints evaluated at randomly sampled scenarios. Under the required convexity, sampling, and solution assumptions, the resulting solution admits a bound on the probability of violating an unseen constraint. Campi, Garatti, and Prandini [11] provided a tutorial treatment of this approach, explaining its algorithmic aspects and applications to systems and control design. For estimation, an approximation-error requirement can be expressed as a sampled constraint on the estimator and its error bound.

PAC model learning has been used to construct numerical approximations for verification. Xue et al. [12] proposed PAC model checking of black-box continuous-time dynamical systems. Their method uses scenario optimization, formulated as a linear program, to learn an approximation of system trajectories together with a bound on the approximation error. The resulting intervals around the learned trajectories support finite-time safety analysis with PAC guarantees. Li et al. [13] used scenario optimization to learn affine approximations with PAC guarantees for analyzing the local robustness of deep neural networks.

Learning methods also construct set-valued approximations of reachable behavior. Fan et al. [14] developed DryVR for systems described by a black-box trajectory simulator and a known transition graph. It learns trajectory sensitivity and combines this information with reachability analysis and compositional reasoning. Sun and Mitra [15] developed NeuReach, which uses neural

networks to learn a reachability function from system executions. The learned function produces reachable-set approximations for initial sets in a parameterized family and supports queries for online monitoring and safe planning.

Scenario optimization has also been used to construct more flexible reachable-set representations. Dietrich et al. [16] studied data-driven estimation of nonconvex reachable sets using scenario optimization. Their two constructions represent a reachable set through a union of partition cells or a sublevel set defined by radial basis functions. The probabilistic bounds are evaluated after constructing the approximation and concern the coverage of the reachable-state distribution by the resulting set.

When a learned dynamical model is used to predict several future states, uncertainty must also be propagated across prediction steps. Polymenakos et al. [17] derived probability bounds for trajectories generated through iterative predictions with Gaussian processes. Their method computes regions around predicted trajectory values and bounds the probability that GP trajectories remain within those regions. It can also incorporate a given control law to analyze closed-loop trajectories of the learned GP model.

Statistical learning and sampled executions also support the verification of system properties. Chen et al. [18] learned a regular model of feasible paths in sequential programs and checked it for incorrect behavior. Their procedure uses sampling in place of exact equivalence checking and provides a PAC correctness guarantee for the learned model. Ashok et al. [19] developed PAC statistical model checking for unbounded reachability in Markov decision processes and stochastic games, including settings with limited knowledge of the transition structure. Jha et al. [20] studied Bayesian statistical model checking, where sampled executions provide evidence about a probabilistic specification.

For control synthesis, Liu et al. [21] studied reinforcement learning for deadline-safe reach-avoid control, combining policy synthesis with safety and timing objectives.

The reviewed methods use statistical guarantees for different purposes, including approximation, verification, and control synthesis. For reliable estimation, the relevant guarantee concerns the error of a learned mapping under a stated input distribution. Its interpretation depends on the sampling assumptions and the procedure used to construct the mapping. Applying such a guarantee within a feedback loop additionally requires examining how the estimates are used during execution.

2.4 Runtime Assurance and the Simplex Architecture

Runtime assurance uses an online decision mechanism to determine when a system should continue normal operation and when a safety response is required. Seto et al. [22] introduced the Simplex architecture for safe online control-system upgrades. The architecture separates an advanced

controller from a safety controller and uses a decision mechanism to transfer control between them. The advanced controller can provide improved performance without being formally verified, while the safety controller and switching mechanism provide the basis for the safety argument.

The switching mechanism must determine when continued use of the advanced controller would prevent safe recovery. Bak et al. [23] used discrete abstractions and model checking to design switching logic for a class of hybrid systems. Their approach automates the construction of the Simplex decision module and verifies its correctness and timing in an off-road vehicle application.

To support switching decisions during execution, Bak et al. [24] studied real-time reachability for verified Simplex design. Their approach combines online and offline reasoning with reachable-state computations. It preserves safety under the model and controller assumptions while reducing the conservatism of earlier switching approaches.

Simplex provides an architectural reference for separating ordinary control from a fallback response. In the framework considered here, the runtime decision concerns the reliability of the feedback information supplied to an existing controller. The specific mode-selection procedure and the conditions for using learned estimates are developed in Chapter 6.

2.5 GPU Power Modeling and Feedback Power Control

Power control regulates server power consumption through adjustments to operating settings. Lefurgy et al. [25] developed server-level feedback power control using measured power to select performance states under a power constraint. Their controller design includes control-theoretic analysis of stability and performance.

At the data-center scale, power capping supports oversubscription of the available power infrastructure. Sakalkar et al. [26] studied power oversubscription using a medium-voltage power plane and priority-aware capping. Kumbhare et al. [27] developed prediction-based oversubscription for cloud platforms, using workload-related predictions to guide resource placement and power management. These methods coordinate computing demand with infrastructure limits and account for the performance requirements of the hosted workloads.

For GPU servers, Ma et al. [28] proposed CapGPU for machine-learning inference workloads running on multiple GPUs and a host CPU. Their work addresses power capping together with inference performance. CapGPU provides the base controller for the GPU power-control application considered in this thesis.

Alongside controller design, GPU power modeling has been studied to predict power consumption and support energy-efficient execution. Hong and Kim [29] developed an integrated model for predicting GPU power consumption and performance. Their approach combines predicted execu-

tion times with an empirical power model and uses the resulting predictions to select the number of active processors that maximizes performance per watt. Bridges et al. [30] reviewed GPU power profiling, modeling, and simulation. Their survey discusses power measurement methods, models based on hardware performance counters, and simulation approaches that predict power consumption from program code and architectural information. More recent work extends power modeling to modern GPU architectures. Kandiah et al. [31] developed AccelWattch, which uses hardware power measurements to calibrate component power models and combines them with component activity information. Depending on the configuration, this activity information comes from hardware performance counters, simulation, or a combination of the two.

Both power-model development and feedback control depend on the quality of the measurements used for calibration, validation, or runtime decisions. Carstens et al. [32] reviewed measurement uncertainty in energy monitoring. In a feedback power-control system, an underestimated reading can lead to operating settings that permit excessive power consumption, while an overestimated reading can cause unnecessary throttling. Power modeling and measurement uncertainty therefore provide the application background for estimating power when the direct reading becomes unreliable.

This thesis studies the PAC properties of piecewise approximation and the construction of estimators from given samples for feedback control, using GPU power control as the application. The following chapter introduces the notation, statistical foundations, and problem formulation.

Chapter 3

Preliminaries and Problem Formulation

This chapter introduces the notation and mathematical foundations for constructing estimators with PAC guarantees. We then formulate the estimation problem for an existing control system and present the Simplex framework for offline estimator construction and online control mode selection.

3.1 Notation

This section introduces the mathematical and statistical notation used throughout this thesis. The set of real numbers is denoted by $\mathbb{R}$, and $\mathbb{R}^d$ represents the d-dimensional Euclidean space. We use lowercase letters for scalars, bold lowercase letters for vectors, and uppercase letters for sets. A column vector $\mathbf{x} = [x_1, \ldots, x_d]^\mathsf{T} \in \mathbb{R}^d$ represents a collection of d variables or features, and x_j denotes its j-th component. For a scalar $x \in \mathbb{R}$, $|x|$ denotes its absolute value. Given a vector $\mathbf{x}$, we use $\|\mathbf{x}\|_\infty$ to denote its infinity norm, i.e., $\|\mathbf{x}\|_\infty = \max_{1 \le i \le d} |x_j|$.

A box X in the Euclidean space $\mathbb{R}^d$ is a bounded region whose coordinates are restricted to fixed intervals along each dimension. In particular, we write $X = [a_1, b_1] \times \cdots \times [a_d, b_d]$, where $a_i, b_i \in \mathbb{R}$ and $a_i \le b_i$ for $i = 1, \ldots, d$. We use such boxes to represent bounded regions of the input space and as the basic domains for constructing estimators from data in later sections.

Let $\mathcal{D}$ denote a probability distribution over the input space. We write $\mathbf{x} \sim \mathcal{D}$ to indicate that $\mathbf{x}$ is drawn according to $\mathcal{D}$. We collect samples $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ independently from $\mathcal{D}$. The resulting samples are independently and identically distributed (i.i.d.), meaning that they are mutually independent and each follows the same distribution $\mathcal{D}$.

We consider an unknown relationship between an input vector and a system state or output. This relationship is represented by $y = \varphi(\mathbf{x})$, where $\varphi : \mathcal{X} \to \mathbb{R}$ denotes an unknown function and $\mathcal{X} \subseteq \mathbb{R}^d$ is the input space. The variable y represents a system state or an observable output that we aim to estimate from the available input information $\mathbf{x}$.

We use $\hat{y} = p(\mathbf{x}, \mathbf{w})$ to denote an estimator of y, where $\mathbf{w}$ is a vector of parameters for the

estimator. The corresponding estimation error is defined as $e(\mathbf{x}, \mathbf{w}) = p(\mathbf{x}, \mathbf{w}) - \varphi(\mathbf{x})$, and its magnitude is $|e(\mathbf{x}, \mathbf{w})| = |p(\mathbf{x}, \mathbf{w}) - \varphi(\mathbf{x})|$.

For a collection of observed input and output pairs, we use $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ to denote a finite dataset, where $y_i = \varphi(\mathbf{x}_i)$ is the corresponding output associated with $\mathbf{x}_i$. We use $x_{i,j}$ to denote the j-th component of the i-th feature sample $\mathbf{x}_i$. The dataset is used to construct an estimator without requiring direct access to the unknown function φ.

Finally, $\Pr(\cdot)$ denotes probability and $\mathbb{E}[\cdot]$ denotes expectation. For an event A under a random input $\mathbf{x} \sim \mathcal{D}$, we write $\Pr_{\mathbf{x}\sim\mathcal{D}}(A)$ to indicate the underlying probability distribution. These notations will be used in the following sections to express PAC guarantees over new inputs and the sampled data used to construct our data-driven estimators.

3.2 Reliable State Estimation

The goal of state estimation is to infer an unknown system state from observed information. Using the relationship introduced in Section 3.1, we formulate this task as approximating $\varphi(\mathbf{x})$ with the estimator $p(\mathbf{x}, \mathbf{w})$. The specific system setting and feedback requirements are described in Section 3.5.

The reliability of an estimator can be characterized by its estimation error. Ideally, this error should remain within a bound throughout the input space. We use $\epsilon \geq 0$ as an optimization variable representing this bound. The estimator and the smallest such bound could be obtained by solving the following optimization problem:

$$\begin{aligned} \min_{\mathbf{w},\,\epsilon} \quad & \epsilon \\ \text{s.t.} \quad & |p(\mathbf{x}, \mathbf{w}) - \varphi(\mathbf{x})| \leq \epsilon, \qquad \forall\, \mathbf{x} \in \mathcal{X}. \end{aligned} \tag{3.2.1}$$

Formulation (3.2.1) captures the desired estimation property: the estimation error should remain below ϵ for every input in the input space $\mathcal{X}$. Solving the above formulation is challenging, since the constraint must hold when the input space $\mathcal{X}$ contains infinitely many possible inputs. In this thesis, we focus on the convex case, where the corresponding constraint function is convex with respect to the optimization variables.

The function φ is assumed to be unknown, and in practice, only a finite set of input and output observations is available. The estimation error can therefore be evaluated at the observed inputs, resulting in a finite set of constraints. However, satisfying these constraints only ensures that the error bound holds at the observed inputs. It does not ensure that the same bound holds for all $\mathbf{x} \in \mathcal{X}$.

This difference between the finite observations and the desired constraint over the entire input space motivates the scenario formulation considered in the next section.

3.3 Scenario Optimization

The formulation in (3.2.1) is an optimization problem with infinitely many constraints. As discussed in Section 3.2, such problems are generally difficult to solve directly because the complete constraint set cannot be explicitly evaluated. Scenario optimization provides a framework for handling this type of problem by replacing the original constraint set with constraints evaluated at a finite number of randomly sampled scenarios. The resulting optimization problem can be solved using the available samples, while the solution can be analyzed through the probability of violating the original constraint [10].

In general, consider an optimization problem of the form

$$\begin{aligned} \min_{\mathbf{z}} \quad & \mathbf{c}^\top \mathbf{z} \\ \text{s.t.} \quad & f_{\mathbf{x}}(\mathbf{z}) \leq 0, \qquad \forall\, \mathbf{x} \in \mathcal{X}. \end{aligned} \tag{3.3.1}$$

Here, $|\mathbf{z}|$ denotes the number of optimization variables, and $\mathbf{c}$ is a fixed objective coefficient vector of the same dimension as $\mathbf{z}$. In this thesis, we consider convex scenario problems. For every fixed $\mathbf{x} \in \mathcal{X}$, the constraint function $f_{\mathbf{x}}(\mathbf{z})$ is continuous and convex with respect to $\mathbf{z}$. The convex scenario theorem used below requires convexity with respect to the optimization variables, but does not require the input domain $\mathcal{X}$ to be convex [10].

Let $\mathbf{x}_1, \ldots, \mathbf{x}_N$ be N independent samples drawn from the distribution $\mathcal{D}$. In the scenario optimization problem, only the constraints corresponding to the sampled inputs are considered. The resulting problem is formulated as

$$\begin{aligned} \min_{\mathbf{z}} \quad & \mathbf{c}^\top \mathbf{z} \\ \text{s.t.} \quad & f_{\mathbf{x}_i}(\mathbf{z}) \leq 0, \qquad \mathbf{x}_i \in \mathcal{X}, \quad i = 1, \ldots, N. \end{aligned} \tag{3.3.2}$$

The scenario problem in (3.3.2) contains only a finite number of constraints. Therefore, compared with the original problem in (3.3.1), it becomes a finite convex optimization problem that can be solved using standard optimization algorithms with available solvers.

However, the solution of the scenario problem is obtained by considering the constraints only at the sampled scenarios. Let $\mathbf{z}^*$ denote the optimal solution obtained from the scenario problem with N sampled scenarios. The satisfaction of the constraints associated with these sampled scenarios does not directly imply satisfaction of the original constraint over the entire input space. The probability that the solution violates the original constraint for an unseen scenario is expressed as

$$\Pr_{\mathbf{x} \sim \mathcal{D}} \left(f_{\mathbf{x}}(\mathbf{z}^*) > 0 \right). \tag{3.3.3}$$

This probability represents the probability that the solution obtained from the sampled scenarios violates the original constraint for an unseen scenario. The scenario approach establishes a relationship between the number of sampled scenarios and this violation probability, which is summarized by the following theorem.

Theorem 3.3.1 (Scenario guarantee [10])**.** *Consider a fixed convex scenario problem with samples drawn independently from the same distribution $\mathcal{D}$. Assume that the sampled problem is feasible and attains a unique optimum with probability one, and that the relevant events are measurable. If multiple optimal solutions exist, a fixed tie-breaking rule satisfying the conditions in Appendix A of [10] is used to select a unique solution. Given a violation parameter $\alpha \in (0, 1)$ and a confidence parameter $\beta \in (0, 1)$, if the number of sampled scenarios satisfies*

$$N \geq \frac{2}{\alpha}\left(\ln\frac{1}{\beta} + |\mathbf{z}|\right), \tag{3.3.4}$$

then, with probability at least $1 - \beta$ over the sampled data, the optimal solution $\mathbf{z}^*$ of (3.3.2) has a violation probability no greater than α, i.e.,

$$\Pr_{\mathbf{x}\sim\mathcal{D}}\left(f_{\mathbf{x}}(\mathbf{z}^*) > 0\right) \leq \alpha. \tag{3.3.5}$$

The confidence level $1 - \beta$ concerns the random dataset used to construct the solution. The violation probability concerns a new input drawn from $\mathcal{D}$ after that solution has been fixed.

Theorem 3.3.1 provides a probabilistic guarantee for the solution obtained from the finite scenario problem. For the state estimation problem considered in this thesis, violation of the constraint means that the estimation error exceeds the error bound ϵ. Therefore, the scenario guarantee can be interpreted as a probabilistic guarantee on the estimation accuracy, which leads to the PAC approximation introduced in the next section.

Zhuo add a figure here to explain the relationship among α, β, N

3.4 Probably Approximately Correct Approximation

Under the conditions of Theorem 3.3.1, scenario optimization can be used to obtain a PAC approximation of an unknown function from a sufficiently large number of samples. In this thesis, the unknown function $\varphi(\mathbf{x})$ represents the relationship between the available system information and the system state or output. The estimator $p(\mathbf{x}, \mathbf{w})$ is constructed from observed samples and is used to approximate this unknown function.

Definition 3.4.1 (PAC $(\epsilon, \alpha, \beta)$ Approximation)**.** *Given a function $\varphi : \mathcal{X} \to \mathbb{R}$, a function p constructed from sampled data, an error bound $\epsilon \geq 0$, a violation parameter $\alpha \in (0, 1)$, and a*

confidence parameter $\beta \in (0,1)$*, we call* p *a PAC* $(\epsilon, \alpha, \beta)$ *approximation of* φ *in a domain* $\mathcal{X} \subseteq \mathbb{R}^d$ *if*

$$\Pr_{\mathbf{x}\sim\mathcal{D}} \left(|p(\mathbf{x}) - \varphi(\mathbf{x})| \leq \epsilon\right) \geq 1 - \alpha \tag{3.4.1}$$

with confidence at least $1 - \beta$ *over the sampled data used to construct* p *and its bound.*

The three parameters in the PAC guarantee describe different aspects of the approximation. The error bound ϵ specifies the tolerance in the approximation guarantee. The violation parameter α specifies the maximum probability that the estimation error exceeds ϵ for an input drawn from the underlying distribution. The confidence parameter β bounds the probability, over the sampled data, that the constructed estimator and its bound fail to satisfy this probabilistic accuracy requirement. Hence, a smaller α requires the estimator to be accurate over a larger fraction of the input distribution, while a smaller β requires a higher confidence in the obtained guarantee.

The PAC definition can be related to the scenario optimization problem by considering an estimator with a parameter vector $\mathbf{w}$. Let $p(\mathbf{x}, \mathbf{w})$ denote a fixed template function whose parameters range over a bounded convex set $\mathcal{W}$. For a fixed input $\mathbf{x}$, the estimation error is given by $|p(\mathbf{x}, \mathbf{w}) - \varphi(\mathbf{x})|$. The corresponding optimization problem seeks an estimator with the smallest error bound that satisfies the error constraint for the available samples:

$$\begin{aligned} \min_{\mathbf{w},\,\epsilon} \quad & \epsilon \\ \text{s.t.} \quad & |p(\mathbf{x}_i, \mathbf{w}) - \varphi(\mathbf{x}_i)| \leq \epsilon, \qquad i = 1, \ldots, N, \\ & \mathbf{w} \in \mathcal{W}, \qquad \epsilon \geq 0. \end{aligned} \tag{3.4.2}$$

If the constraint function in (3.4.2) is convex with respect to the optimization variables for every fixed scenario $\mathbf{x}_i$, the problem is a convex scenario optimization problem. Under the assumptions of the scenario approach, a sufficiently large number of independently and identically distributed samples is then sufficient to provide a PAC guarantee for the resulting estimator.

For the estimator considered above, the number of samples required for the guarantee can be bounded in terms of the violation and confidence parameters.Let $|\mathbf{w}|$ denote the number of parameters in the estimator. The optimization variables consist of $\mathbf{w}$ and the error bound ϵ, so their total number is $|\mathbf{w}| + 1$. If

$$N \geq \frac{2}{\alpha}\left(\ln\frac{1}{\beta} + |\mathbf{w}| + 1\right) = N_{\alpha,\beta}, \tag{3.4.3}$$

then, under the assumptions of the scenario theorem, the solution $(\mathbf{w}^*, \epsilon^*)$ of (3.4.2) yields a PAC $(\epsilon^*, \alpha, \beta)$ approximation of φ.

The sample bound in (3.4.3) provides a sufficient sample size for obtaining the PAC guarantee under the theorem's conditions. It does not determine the value of the optimized error bound ϵ^*.This bound is obtained by solving (3.4.2) and depends on the chosen template, the unknown

function, and the sampled data. The sampling distribution affects where the observations are collected. Therefore, using the same domain, PAC parameters, and sample size can produce different approximations and optimized error bounds.

The PAC guarantee therefore provides a probabilistic formulation of reliable state estimation. Instead of requiring the estimation error to be bounded for every possible input, the estimator is required to satisfy the error bound with high probability over the input distribution and with high confidence over the sampled data. The next section specifies the system setting and the estimation requirement to which this guarantee will be applied.

Zhuo Add Example 1 here. Use the same function, domain, template, PAC parameters, and sample size under different sampling distributions(gaussian and uniform). Compare the fitted models and error bounds to illustrate the role of the sampling distribution.

Zhuo Add a remark after Example 1. Explain that the PAC guarantee concerns the sampling distribution. The same domain can support different distributions, and a guarantee under one distribution does not directly establish the same guarantee under another.

3.5 Problem Formulation

We consider a system operating in discrete time with an existing feedback controller. At each time step, feedback is obtained and a control input is computed and applied to the system. This input affects the system's behavior at the next time step. The framework requires neither a known model of the system dynamics nor access to the controller's internal design. It uses the controller through its input and output interfaces.

In this setting, y is the true state component or output required for feedback, and $\tilde{y}$ is its direct measurement. In the absence of measurement error, $\tilde{y} = y$. We consider sensing errors that can occur intermittently or persist across consecutive time steps. Their magnitudes and occurrence times are not known in advance, and no particular probability distribution is imposed on them.

The feature vector $\mathbf{x}$ consists of other measured system variables and does not use $\tilde{y}$. Estimating a target quantity from other measured information is an established approach in virtual sensing [33]. For the faults considered here, we assume that these features remain reliably measured and contain sufficient information to support $y = \varphi(\mathbf{x})$. This relationship is taken to remain valid within the considered operating domain during data collection and online use. Feature measurement errors are not explicitly modeled.

The dataset $\mathcal{S}$ is collected through independent system runs using the same randomized initialization and observation procedure, with one sample from each run. The feature samples are therefore i.i.d. according to the distribution $\mathcal{D}$ induced by this procedure. Target values are obtained under reliable measurement conditions and treated as $y_i = \varphi(\mathbf{x}_i)$. This collection procedure

does not imply that consecutive states during online operation are independent.

We assume that the existing controller meets the prescribed control and safety requirements under accurate feedback and the considered operating conditions. A conservative controller is also available. Within its specified operating region, it maintains safe operation without relying on $\tilde{y}$, possibly at reduced performance. These properties are conditions for applying the framework.

Given $\mathcal{S}$, $\mathcal{D}$, α, β, and an application threshold $\epsilon_{\min} > 0$, the estimation objective is to construct a PAC approximation with an error bound no greater than $\epsilon_{\min}$. For a bound obtained from (3.4.2), this requires $\epsilon^* \leq \epsilon_{\min}$. The construction must identify the input domains on which the required guarantee has been established and report when it is unavailable. A local guarantee refers to the corresponding conditional input distribution. An overall guarantee requires a separate analysis of the combined estimator. Estimation guarantees characterize approximation error. Safety during system execution also depends on the use of these estimates by the controller and is assessed separately.

3.6 Framework Overview

Figure 3.6.1 shows the Simplex control framework. Following the Simplex architecture, a decision module coordinates the existing controller and a conservative controller [34]. Our framework also selects whether the existing controller receives measured or estimated feedback.

3.6.1 Offline Construction and Evaluation

The offline stage constructs the estimator and records its error bounds and domains of validity. It also evaluates system executions using estimated feedback. The resulting limit L_{safe} specifies the number of consecutive time steps for which this feedback may be used before conservative control is required. The limit is evaluated for the particular estimator, controller, and operating conditions. Its evaluation procedure and statistical interpretation are developed in the chapter on online control.

3.6.2 Online Estimation and Control Mode Selection

At each time step, the framework obtains $\tilde{y}$ and $\mathbf{x}$ and computes $\hat{y} = p(\mathbf{x}, \mathbf{w})$. The decision module uses the applicable error guarantee, the comparison between $\tilde{y}$ and $\hat{y}$, and the duration of consecutive use of estimated feedback to select a control mode. In measurement based control, the existing controller receives $\tilde{y}$. In estimation based control, it receives $\hat{y}$. Conservative control uses the fallback controller when the estimation requirements or duration restriction prevent continued use of the other modes. The selected controller produces the control input $\mathbf{u}$, which affects the next

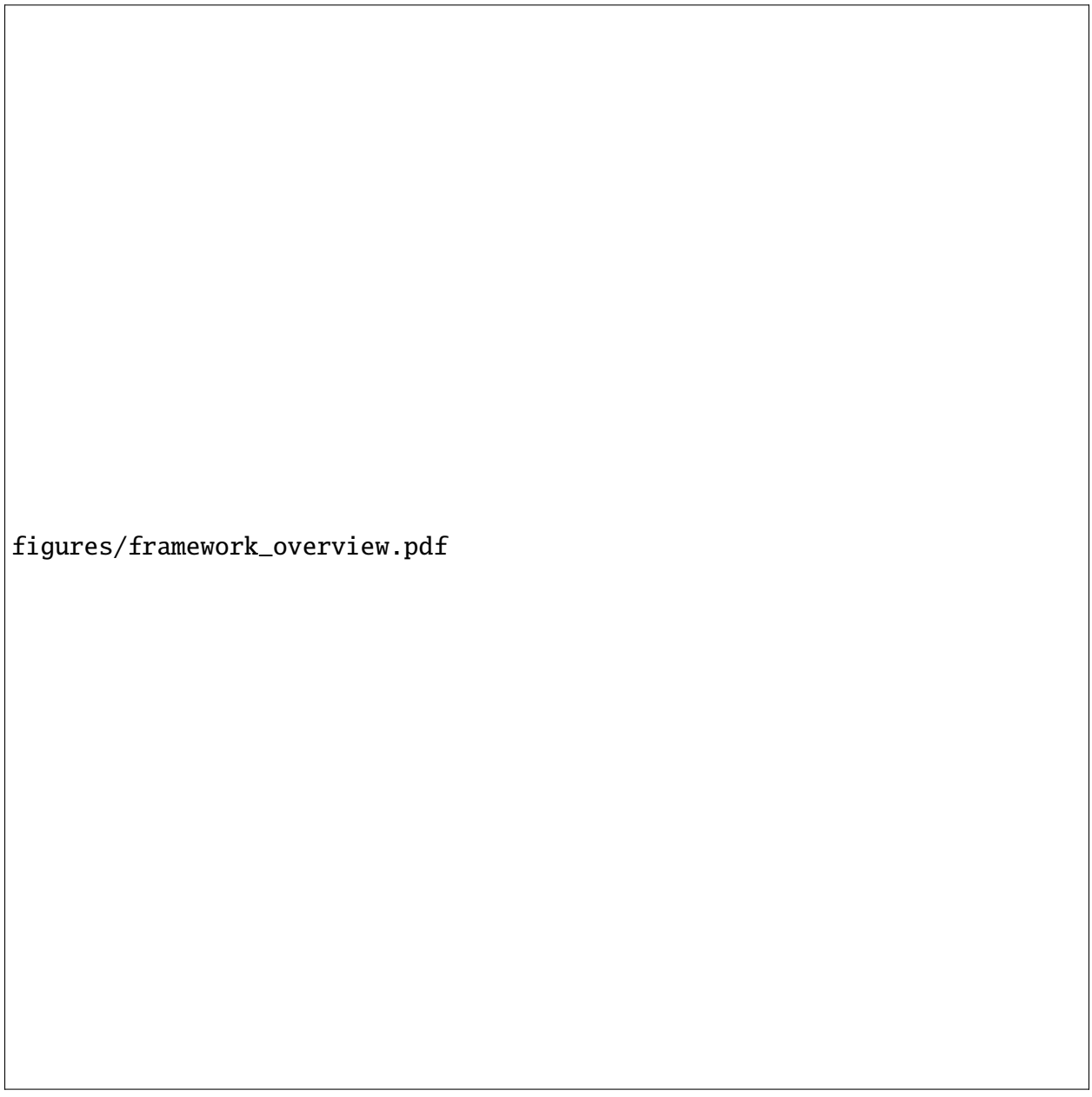


Figure 3.6.1: Overview of the Simplex control framework.

time step. The next chapter develops the construction of PAC state estimators. The subsequent chapter presents the control mode selection rules and the evaluation of system safety.

Zhuo add a description that this method can be easily added to other controller

Chapter 4

Piecewise PAC Approximation

We seek to approximate an unknown function from samples and establish a PAC guarantee for the approximation error, as formulated in Chapter 3. A single approximation over the input domain may not meet the required accuracy. We therefore consider a piecewise approximation, which divides the input domain into smaller regions and uses a separate approximation function in each region. This chapter studies whether a PAC guarantee can be established within each region and whether the combined approximation admits a PAC guarantee over the entire input domain.

Let $X \subseteq \mathcal{X}$ be the bounded box considered for approximation, and let $X_1, \ldots, X_K$ be sub-boxes forming a subdivision of X. Each region X_k is associated with an approximation function p_k and an error bound $\epsilon_k \geq 0$. The piecewise approximation is represented by

$$\mathcal{R} = \{(X_k, p_k, \epsilon_k)\}_{k=1}^{K}, \qquad \bigcup_{k=1}^{K} X_k = X.$$

The functions p_k approximate the same unknown function φ on its respective region X_k. The combined approximation uses the corresponding local function to evaluate an input in each region. The local PAC requirement is defined as follows.

Definition 4.0.1 (Piecewise PAC Approximation)**.** *Let $\mathcal{D}$ be the input distribution over X, and the PAC parameter $\alpha, \beta \in (0, 1)$. We call the collection $\mathcal{R}$ a piecewise PAC approximation if, for every region X_k with $\Pr_{\mathbf{x}\sim\mathcal{D}}(\mathbf{x} \in X_k) > 0$, the local requirement*

$$\Pr_{\mathbf{x}\sim\mathcal{D}} \left(|p_k(\mathbf{x}) - \varphi(\mathbf{x})| \leq \epsilon_k \,|\, \mathbf{x} \in X_k\right) \geq 1 - \alpha$$

holds with confidence at least $1 - \beta$ over the sampled data used in the construction. The confidence requirement applies to each local statement individually.

The local guarantee concerns the input distribution conditioned on $\mathbf{x} \in X_k$. The overall guar-

antee concerns the combined approximation under $\mathcal{D}$ over X. The following sections examine the conditions under which the regional approximations satisfy this property and how their guarantees can be combined. The specific function templates and estimator construction are developed in Chapter 5.

4.1 PAC Guarantees under Regional Resampling

4.1.1 Regional Sampling Mechanism

Zhuo references: look into regional resampling methods and their sampling mechanisms with guarantee formulations.

4.1.2 Local and Overall Guarantees

Zhuo state the local and overall guarantees for the selected resampling references

4.2 PAC Guarantees from a Fixed Dataset

4.2.1 Approximation on a Smaller Region

We first consider an input region specified independently of the samples used to construct its approximation. Under the assumptions of Theorem 3.3.1, an approximation obtained from the fixed convex scenario formulation with a sufficient number of i.i.d. samples from the same distribution has a PAC guarantee on that region.

4.2.2 Approximation on Entire Regions

4.2.3 Approximation on Sample-Insufficient Regions

Chapter 5

Adaptive Piecewise Polynomial Estimator Construction

Chapter 4 discusses the PAC properties of piecewise approximation. This chapter describes how to construct a piecewise polynomial estimator from the given dataset. Each region uses a polynomial whose coefficients are determined from the samples in that region. The construction starts with a linear approximation. If its error exceeds the required threshold, we first consider increasing the polynomial degree. Subdivision provides further refinement when degree selection does not meet the threshold.

We first introduce the polynomial representation and explain how to determine the coefficients of a linear approximation from the samples. We then present adaptive degree selection, impurity-based subdivision with the complete construction algorithm.

5.1 Polynomial Representation

The polynomial template specifies the monomials whose coefficients are to be fitted. We first describe its general form, then specialize it to a linear function for each region of a piecewise approximation.

Definition 5.1.1 (Polynomial Template [35])**.** *Given an input vector* $\mathbf{x} = [x_1, \ldots, x_d]^\top \in \mathbb{R}^d$*, a monomial is a product of nonnegative integer powers of the input components. It is represented by a* d*-tuple* $\mathbf{q} = (q_1, \ldots, q_d)$ *as* $\mathbf{x}^{\mathbf{q}} = x_1^{q_1} \cdots x_d^{q_d}$*, where* q_j *is a nonnegative integer specifying the exponent of* x_j*. The total degree of this monomial is* $q_1 + \cdots + q_d$*.*

A polynomial template function p *is a finite linear combination of distinct monomials. For a finite collection of exponent tuples, a real coefficient* $w_{\mathbf{q}}$ *is assigned to each corresponding*

monomial. The template is written as

$$p(\mathbf{x}, \mathbf{w}) = \sum_{\mathbf{q}} w_{\mathbf{q}} \mathbf{x}^{\mathbf{q}} = \sum_{\mathbf{q}} w_{\mathbf{q}} x_1^{q_1} \cdots x_d^{q_d}, \tag{5.1.1}$$

where the sum runs over the selected exponent tuples, and $w_{\mathbf{q}} \in \mathbb{R}$ *is the component of* $\mathbf{w}$ *associated with the monomial* $\mathbf{x}^{\mathbf{q}}$.

We denote the highest total degree among the monomials included in the template by m, defined as

$$m = \max_{\mathbf{q}} (q_1 + \cdots + q_d). \tag{5.1.2}$$

When $m = 1$, the template contains only constant and linear terms, giving an affine function of $\mathbf{x}$. We refer to this case as a linear approximation and present it in the following subsection.

5.1.1 Piecewise Linear Approximation

A piecewise linear approximation uses a linear function in each region of the input domain. Different regions may use different coefficients, allowing the approximation to describe different local relationships between the features and the target while retaining a simple function within each region.

The local linear function is obtained from (5.1.1) by setting $m = 1$. The exponent tuples then satisfy $q_1 + \cdots + q_d \leq 1$, corresponding to the monomials $1, x_1, \ldots, x_d$. Including all these terms gives

$$p(\mathbf{x}, \mathbf{w}) = w_0 + \sum_{j=1}^{d} w_j x_j, \qquad \mathbf{w} = [w_0, w_1, \ldots, w_d]^\mathsf{T}, \tag{5.1.3}$$

where the coefficients $w_{\mathbf{q}}$ are indexed as $w_0, \ldots, w_d$ for this template.

For the regions X_k considered in Chapter 4, the corresponding local functions have the form

$$p_k(\mathbf{x}) = p(\mathbf{x}, \mathbf{w}_k) = w_{k,0} + \sum_{j=1}^{d} w_{k,j} x_j, \qquad \mathbf{x} \in X_k, \tag{5.1.4}$$

where $\mathbf{w}_k$ contains the coefficients associated with region X_k. Thus, the same linear template is used throughout the piecewise approximation, while its coefficients depend on the region.

To fit a local model on a given box $X' \subseteq X$, we use the N' available samples in that box,

substituting (5.1.3) into (3.4.2) gives

$$\begin{aligned} \min_{\mathbf{w},\,\epsilon} \quad & \epsilon \\ \text{subject to} \quad & -\epsilon \le w_0 + \sum_{j=1}^{d} w_j x_{i,j} - y_i \le \epsilon, \quad i = 1, \ldots, N', \\ & \mathbf{w} \in \mathcal{W}, \quad \epsilon \ge 0. \end{aligned} \tag{5.1.5}$$

For each fixed sample, the predicted value is linear in the unknown coefficients. The two inequalities bound the positive and negative approximation errors, respectively. Together with the linear inequalities defining the bounded coefficient domain $\mathcal{W}$, they make (5.1.5) a linear program with $d + 2$ optimization variables, consisting of $d + 1$ coefficients and the error bound ϵ.

Let ϵ^* be the optimal error bound obtained from (5.1.5). This selection leaves the optimal error bound unchanged and determines a unique coefficient vector. The resulting model $p(\mathbf{x}, \mathbf{w}^*)$ minimizes the maximum absolute error over the samples used in the fit among the linear models allowed by $\mathcal{W}$. If this bound exceeds $\epsilon_{\min}$, we next consider whether a higher-degree polynomial can meet the threshold in the same region. The following section describes this degree selection procedure.

5.2 Adaptive Degree Selection

When the fitted error of a linear model exceeds $\epsilon_{\min}$, we can try a higher-degree polynomial in the same region. Additional monomials allow the model to represent nonlinear dependence on the input features. Error-guided degree refinement is also considered in [36]. In our construction, the violation parameter α and confidence parameter β are prescribed before construction and remain fixed. The degree search uses the same samples in the current region. It changes only the template degree and refits the coefficients.

For a template degree $m \ge 1$, we include all monomials whose total degree is at most m:

$$p_m(\mathbf{x}, \mathbf{w}) = \sum_{q_1 + \cdots + q_d \le m} w_{\mathbf{q}} \mathbf{x}^{\mathbf{q}}, \tag{5.2.1}$$

where the sum runs over tuples of nonnegative integers. This template contains

$$M_m = \binom{d + m}{m} \tag{5.2.2}$$

coefficients. At $m = 1$, it reduces to the linear template in (5.1.3). Here, m is the maximum total degree allowed by the template.

Consider a box $X' \subseteq X$ containing N' samples. For a chosen degree m, its polynomial is fitted by solving

$$\begin{aligned} \min_{\mathbf{w},\,\epsilon} \quad & \epsilon \\ \text{subject to} \quad & -\epsilon \le p_m(\mathbf{x}_i, \mathbf{w}) - y_i \le \epsilon, \quad i = 1, \ldots, N', \\ & \mathbf{w} \in \mathcal{W}_m, \quad \epsilon \ge 0. \end{aligned} \tag{5.2.3}$$

The coefficient domain $\mathcal{W}_m$ is a nonempty bounded convex set defined by linear inequalities. At each sample, the monomial values are known. The constraints are therefore linear in the coefficients and ϵ, so the problem is an LP with $M_m + 1$ optimization variables. We use the coefficient selection rule in Section 5.1.1 and denote the solution by $(\mathbf{w}_m^*, \epsilon_m^*)$.

Increasing the degree does not increase this optimal sample error when the coefficient constraints permit the previous fit. For example, the quadratic model $a + bx + cx^2$ reproduces the linear model $a + bx$ by setting $c = 0$. It then gives exactly the same predictions and sample error. We allow this choice by keeping the bounds on existing coefficients and choosing bounds that contain zero for each new coefficient. The following lemma states the resulting error property.

Lemma 5.2.1. *Consider the fitting problems* (5.2.3) *at degrees m and $m + 1$ in the same region, using the same samples. Suppose that every coefficient vector in $\mathcal{W}_m$ belongs to $\mathcal{W}_{m+1}$ after zero coefficients are assigned to the additional monomials. If both problems are solved to optimality, their fitted error bounds satisfy*

$$\epsilon_{m+1}^* \le \epsilon_m^*. \tag{5.2.4}$$

Proof. **Zhuo** add the proof here □

Lemma 5.2.1 concerns the maximum absolute error on the samples used for fitting. The improvement need not be strict. It does not state that the error at unseen inputs decreases with the degree.

The cost of increasing the degree is reflected in the variable count. For fixed input dimension d,

$$M_m = \frac{(m+1)(m+2)\cdots(m+d)}{d!}, \tag{5.2.5}$$

which is a polynomial of degree d in m. Increasing the variable count also increases the sufficient sample requirement from Chapter 3. Substituting $M_m + 1$ into (3.4.3) gives

$$N_{\alpha,\beta}^{(m)} = \left\lceil \frac{2}{\alpha} \left(\ln \frac{1}{\beta} + \binom{d+m}{m} + 1 \right) \right\rceil. \tag{5.2.6}$$

Before fitting degree m, we check whether $N' \ge N_{\alpha,\beta}^{(m)}$. For fixed α and β, the required count increases with m. A region may therefore have enough samples for a linear fit but too few for the next degree. In that case, the search retains the last fitted model and stops.

We also prescribe a maximum degree $m_{\max} \geq 1$ to limit model complexity. Algorithm 1 summarizes the search within a region X'. It starts at $m = 1$ and checks the samples before each fit. After fitting, it compares the error bound with $\epsilon_{\min}$. If the threshold is met, the model is returned. Otherwise, the search tries the next degree until it reaches $m_{\max}$ or the samples are insufficient. The region and its samples remain unchanged throughout the search.

Algorithm 1 Degree selection in a fixed region

Input: Samples $\mathcal{S}'$ in X', error threshold $\epsilon_{\min}$, PAC parameters α, β, maximum degree $m_{\max}$.
Output: Best fitted polynomial approximation p

for $m = 1, \ldots, m_{\max}$ **do**
  if the samples are insufficient for degree m **then**
    if $m = 1$ **then**
      return None
    else
      return the model p_{m-1} fitted in a lower degree
    end if
  end if
  Fit the degree-m polynomial by solving the problem in 5.2.3 using the samples in $\mathcal{S}'$.
  if the fitted error bound is at most $\epsilon_{\min}$ **then**
    return p_m
  end if
end for
return None

The search performs at most $m_{\max}$ fits. If the error still exceeds $\epsilon_{\min}$ at $m_{\max}$, the construction considers subdivision as described in Section 5.3.

5.3 Impurity-Based Subdivision

If the model returned by Algorithm 1 has an error exceeds $\epsilon_{\min}$, a subdivision is considered in its region to get a more accurate piecewise approximation. We select a split based on the impurity measurement described in the following sections. Different impurity measures can produce different subdivisions of the same region.

5.3.1 Residuals and Impurity Criterion

For a current box $X' \subseteq X$, let

$$\mathcal{S}' = \{(\mathbf{x}_i, y_i) \in \mathcal{S} \mid \mathbf{x}_i \in X'\}, \qquad N' = |\mathcal{S}'|.$$

Let p be the model returned by Algorithm 1 for this box. If the algorithm returns None, the box inherits its parent model and is not further refined. Otherwise, we compute the absolute residual of each sample

$$r_i = |p(\mathbf{x}_i, \mathbf{w}^*) - y_i|, \qquad i = 1, \ldots, N'.$$

Residual-range splitting. The objective is to divide X' into two boxes that can be approximated more accurately by separate polynomials. To find such a subdivision, we choose a split that minimizes the following impurity,

$$\max_{k\in\{1,2\}} \left\{ \max_{\mathbf{x}_i,\mathbf{x}_j \in X'_k} |r_i - r_j| \right\}. \tag{5.3.1}$$

The motivation is to group samples with similar approximation errors under the current polynomial p. The same model may fit some parts of X' well and other parts poorly. Minimizing (5.3.1) favors a subdivision with similar residuals within each box. The samples in the two boxes can then be fitted independently. Each polynomial can be adapted to its own group of samples, which by Lemma 5.3.1, improve the approximation compared with using one model over the whole region.

Maximum-residual splitting. The second method evaluates a candidate split using the maximum residual across the two sub-boxes:

$$\max_{k\in\{1,2\}} \left\{ \max_{\mathbf{x}_i \in X'_k} r_i \right\}$$

The residuals are computed from the current polynomial before the sub-boxes are fitted separately. This method provides a baseline for comparison with the residual-range method.

Zhuo Provide an intuitive example or diagram to illustrate the differences between these two methods.

5.3.2 Subdivision Procedure

Searching over all possible split positions is computationally expensive. Instead, we sort the samples along each input dimension and place a candidate split halfway between each pair of adjacent distinct coordinates. With N' samples in the current box, there are at most $N' - 1$ candidates per dimension, yielding $O(dN')$ candidates in total.

We first compute the residual of each sample using the model returned from the Algorithm 1. We then evaluate the candidate split positions with the chosen impurity measure and select those with the smallest score among candidates satisfying the sample-count conditions below.

Several positions may obtain the same minimum score. In this case, we choose the most balanced split by minimizing $|N'_1 - N'_2|$, where N'_1 and N'_2 are the child samples in the subdivisions. The sample counts are considered in two cases:

- If $N' \geq 2N_{\alpha,\beta}$, both children must contain at least $N_{\alpha,\beta}$ samples so that both can be fitted separately.
- If $N' < 2N_{\alpha,\beta}$, one child must contain $N_{\alpha,\beta}$ or more samples. Among the minimum-score candidates, the balance rule favors a split with exactly $N_{\alpha,\beta}$ samples in that child whenever such a split is available.

If no split satisfies these conditions, the current box and its model are retained without further subdivision.

After subdivision, each child is fitted using Algorithm 1. A child with insufficient samples for the initial fit inherits its parent model, as discussed in Section 4.2.3. During estimation, an input on a shared boundary is evaluated using the model with the smallest fitted error bound among the regions containing that input. The following lemma describes the error property of subdivision when the same polynomial template is used for the parent and child fits.

Lemma 5.3.1. *Let* $(\mathbf{w}^*, \epsilon^*)$ *be an optimal solution of* (5.2.3) *at degree* m *using the samples in a parent box* X'*. Suppose that* X' *is divided into two child boxes* X'_1 *and* X'_2*. The PAC approximation in each of the subdivisions* X'_1 *and* X'_2 *has an approximation error which is no greater than* ϵ^**.*

Proof. **Zhuo** adding proof here □

Remark. The impurity criterion 5.3.1 plays a role similar to that in decision tree construction [37]. Here we consider to perform an axis-aligned split classification of the samples to make the samples in each class better approximated by a polynomial function.

Algorithm 2 presents the complete adaptive subdivision procedure. For each box taken from the queue Q, it uses Algorithm 1 to compute a polynomial approximation (Line 5). A box whose fitted error is no greater than $\epsilon_{\min}$ is retained without further subdivision (Line 7). Otherwise, it is split using the rules described above (Line 9). If a child has fewer than $N_{\alpha,\beta}$ samples, the approximation computed for its parent is retained (Line 12). Children with sufficient samples are added to Q for further fitting and subdivision (Line 14).

Theorem 5.3.2. *Algorithm 2 always terminates after finitely many processing steps and returns a piecewise PAC abstraction of* φ*.*

Algorithm 2 Adaptive subdivision algorithm

Input: samples $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, initial box X, error threshold $\epsilon_{\min}$, PAC parameters α, β, maximum degree $m_{\max}$,

Output: Piecewise polynomial PAC abstraction

$\mathcal{R} \leftarrow \emptyset$; # result of the algorithm
Q.enqueue(X); # queue for the unprocessed box
while Q is not empty **do**
  $X' = Q$.dequeue();
  Apply Algorithm 1 to the samples in X' and compute a PAC approximation p w.r.t α, β;
  if the error of the obtained model, ϵ, is no greater than $\epsilon_{\min}$ **then**
    $\mathcal{R} \leftarrow \mathcal{R} \cup (X', p, \epsilon)$;
  else
    Subdivide X' to X'_1, X'_2 based on the error of each sample in X';
    for $i = 1, 2$ **do**
      if the number of samples in X'_i is less than $N_{\alpha,\beta}$ **then**
        $\mathcal{R} \leftarrow \mathcal{R} \cup (X', p, \epsilon)$;
      else
        Q.enqueue(X'_i);
      end if
    end for
  end if
end while
return $\mathcal{R}$;

Proof. **Zhuo** adding proof here □

Termination does not require every retained region to meet $\epsilon_{\min}$. A region may be retained because it has insufficient samples for a separate fit or because no admissible split is available.

Chapter 6

Online Control and Mode Selection

As described in Section 3.6, the online decision maker runs synchronously with the existing controller. At each control period, it selects use the original controller with the direct system measurement, provide a predicted value to that controller, or switch to conservative control. This chapter explains how these decisions are made using the estimator constructed in Chapter 5. We then explain the need for the safe prediction horizon L_{safe} and the role of conservative control. Finally, we describe the offline procedure for estimating L_{safe}.

6.1 Online Decision Making

The decision maker shown in Figure 3.6.1 selects a control mode using the given inputs and the uncertain output measurement. It determines whether to provide the direct measurement or the predicted value to the original system controller. When conservative control is selected, the conservative controller computes the control input instead. The framework can therefore be deployed as an add-on to dynamic systems with different types of controllers. This requires neither a model of the system dynamics nor knowledge of the original controller.

We first describe how the piecewise estimator computes the predicted value and its associated error bound from the current inputs. We then explain how the decision maker uses these quantities to select a control mode.

Given the current feature vector $\mathbf{x}$ and the result piecewise polynomial PAC abstraction $\mathcal{R}$, we identify all the pieces whose regions contain $\mathbf{x}$. Among these pieces, we select the model p_k with the smallest error bound ϵ_k. The predicted value is $\hat{y} = p_k(\mathbf{x})$, and its associated error bound is $\epsilon = \epsilon_k$. The following paragraph explains how these quantities are used to select a control mode.

Control Mode Selection. Algorithm 3 describes the decision-making procedure for each control period. The decision maker first checks whether prediction-based control has been used for the past

L_{safe} consecutive periods (Line 1). If so, it selects conservative control. Otherwise, it obtains the current features $\mathbf{x}$ and the measurement $\tilde{y}$. It then selects the model with the smallest error bound among the pieces containing $\mathbf{x}$ (Line 6). If no such piece exists or the selected error bound exceeds $\epsilon_{\min}$, it selects conservative control (Line 7).

All approximations in $\mathcal{R}$ satisfy the PAC guarantee defined in earlier chapter. When a model is available and $\epsilon \leq \epsilon_{\min}$, it also meets the required prediction accuracy. Its prediction can therefore serve as a reference for assessing the measurement. The decision maker computes $\hat{y} = p(\mathbf{x})$ and checks whether

$$|\tilde{y} - \hat{y}| \leq \epsilon. \tag{6.1.1}$$

If this condition holds, the measurement is considered consistent with the PAC prediction and is trusted as the input to the original system controller. Otherwise, the measurement is treated as unreliable because its difference from the prediction exceeds the current piece's error bound. The decision maker then replaces the measurement with $\hat{y}$, switching to prediction-based control mode. This comparison corresponds to Line 11. In both cases, the original controller computes the control input $\mathbf{u}$.

Algorithm 3 Decision making in a control period

Input: Piecewise approximation $\mathcal{R}$, error threshold $\epsilon_{\min}$, safe prediction horizon L_{safe}, and previous control modes;
Output: Selected control mode and control input $\mathbf{u}$.

1: **if** prediction-based control was used in the past L_{safe} consecutive periods **then**
2: Use the conservative controller strategy;
3: **else**
4: Get the current features $\mathbf{x}$;
5: Obtain the direct measurement $\tilde{y}$;
6: Find a PAC approximation p with the smallest error bound ϵ among all pieces containing $\mathbf{x}$;
7: **if** there is no such pieces available or $\epsilon > \epsilon_{\min}$ **then**
8: Use the conservative control mode;
9: **else**
10: Compute $\hat{y} = p(\mathbf{x})$;
11: **if** $|\tilde{y} - \hat{y}| \leq \epsilon$ **then**
12: Give $\tilde{y}$ to the original system controller;
13: **else**
14: Give $\hat{y}$ to the original system controller;
15: **end if**
16: **end if**
17: **end if**

6.2 Safe Prediction Horizon and Conservative Control

The duration check in Algorithm 3 uses L_{safe} to limit consecutive use of predictions. We explain this limit and the role of conservative control before describing how L_{safe} is estimated offline.

Safe prediction horizon. PAC accuracy is a statistical guarantee. Under the PAC guarantee, the prediction error may exceed ϵ on a set of inputs with probability at most α. Checking $\epsilon \leq \epsilon_{\min}$ only confirms that the model's error bound meets the required accuracy, but does not eliminate the possibility of these prediction errors. When the original controller uses $\hat{y}$, a prediction error may affect the control input $\mathbf{u}$ and lead to unsafe system behavior. We therefore limit the number of consecutive periods using prediction-based control. We use L_{safe} as the maximum permitted number of consecutive prediction-based control periods. After L_{safe} consecutive periods, the decision maker selects conservative control for the next period.

Conservative control. Conservative control uses the fallback controller assumed in Section 3.5. It maintains safe operation within its specified operating region without relying on the potentially faulty target measurement. Depending on the application, it may use other reliably measured state variables or a prescribed control action. For power capping, it can lower operating frequencies and voltages to limit power consumption. Other dynamic systems require a conservative strategy suited to their safety constraints. The resulting loss of performance is accepted to maintain safe operation.

6.2.1 Estimation of L_{safe}

We estimate L_{safe} through several executions from the system in which the original controller continuously uses predictions from $\mathcal{R}$. The purpose is to determine how many consecutive prediction-based control periods the system can sustain safely. The predictor and controller are fixed throughout this evaluation.

Before testing, we specify a safety condition that can be checked using reliable monitoring. An execution is safe for L periods if this condition holds throughout those periods. For power capping, this means that the actual power remains below a certain safety threshold. Other dynamic systems can use their relevant state or output constraints.

For each run, we randomly select an initial state according to $\mathcal{D}$. The initial states selected across runs are independent and identically distributed. We initialize the system at the selected state and run the original controller using only prediction-based control. Let π denote an execution generated in this way.

Algorithm 4 presents the offline estimation procedure. Instead of updating the Bayes factor at each iteration to decide when to stop testing [20], we precompute the required number of

Algorithm 4 Offline estimation of L_{safe}

Input: Initial state distribution $\mathcal{D}$, upper bound $L_{\max}$, and PAC parameters η_1, η_2.
Output: L_{safe}

1: $N_{\text{test}} = \lceil \log \eta_2 / \log(1 - \eta_1) \rceil$;
2: $L_{\text{safe}} = L_{\max}$;
3: **for** $i = 1, \ldots, N_{\text{test}}$ **do**
4: Randomly select an initial state with respect to $\mathcal{D}$;
5: Run the existing system controller using only prediction-based mode for up to L_{safe} periods. We denote this execution by π;
6: Set l to be the index (starting from 1) of the first unsafe period of π;
7: **if** π is not completely safe **then**
8: $L_{\text{safe}} = \min(L_{\text{safe}}, l - 1)$;
9: **end if**
10: **end for**
11: **return** L_{safe};

i.i.d. executions. For the prescribed violation parameter $\eta_1 \in (0, 1)$ and confidence parameter $\eta_2 \in (0, 1)$, this number is

$$N_{\text{test}} = \left\lceil \frac{\log \eta_2}{\log(1 - \eta_1)} \right\rceil. \tag{6.2.1}$$

We begin with $L_{\text{safe}} = L_{\max}$ (Line 2). Each run uses prediction-based control for up to the current candidate horizon (Line 5). If the first unsafe period is detected at index l, we reduce L_{safe} to $l - 1$ (Lines 6–8). Otherwise, L_{safe} remains unchanged. This update ensures that every evaluated execution is safe for at least the first L_{safe} periods. The procedure stops after N_{test} runs and returns the resulting horizon. Similar sampling ideas are used in statistical verification and probabilistic robustness analysis [38, 39]. With this number of independent runs, the returned L_{safe} satisfies

$$\Pr\big(\pi \text{ remains safe for } L_{\text{safe}} \text{ periods}\big) \geq 1 - \eta_1, \tag{6.2.2}$$

with confidence at least $1 - \eta_2$ over the offline runs.

Chapter 7

Experiments

7.1 GPU Power Control Case Study

A GPU server typically uses a feedback controller that periodically reads power from a power meter and adjusts GPU and CPU voltages and frequencies, as well as fan speeds, to maintain power consumption around a desired set point (Section 2.5). We assume that the existing controller provides safe power regulation when the power meter is accurate. Aging and environmental disturbances can, however, introduce errors into the readings. An underestimated reading may cause the controller to increase power beyond the safety threshold, while an overestimated reading may lead to unnecessary throttling and reduce application performance. Our goal is to improve the resilience of the existing controller to power meter errors.

To support control under these errors, we estimate current power from other system features that are assumed to remain reliably measured. This case study examines the resulting power estimators and their use in maintaining power regulation when direct power measurements become unreliable.

7.1.1 Experiment Setup

We use two software simulators and a GPU server, each evaluated with one, two, and four GPUs. Simulator I provides a simple linear environment in which the number of GPUs can be changed easily. Simulator II uses a neural network trained on hardware data and allows repeated experiments and long executions to run more quickly than on the GPU server. The GPU server experiments use actual workloads and include hardware execution overheads.

For Simulator I and the GPU server, we collect trustworthy data through independent runs. For each GPU configuration, we select an initial state uniformly from the safe space and run the simulator or server for one control period. We then record the selected features $\mathbf{x}$ and actual power P as one sample $(\mathbf{x}, P)$. On the server, we obtain P from a reliable power meter, called the monitor.

We repeat these steps using the same distribution for the initial states in every run and assume that the resulting samples are i.i.d.

We use the samples from Simulator I to construct its power estimators. The hardware samples are used to construct the GPU server estimators and to train the neural network for Simulator II. The network learns the relationship between features and power recorded at the same time. Once trained, the network is fixed and used to generate a separate set of samples for constructing Simulator II's estimators. All estimator variants for a given platform and GPU configuration use the same fixed dataset. We give the features, sampling details, and sample counts below when introducing each platform.

Using these datasets, we construct estimators with maximum degrees $m_{\max} = 1$ and $m_{\max} = 3$. The former gives a piecewise linear estimator, while the latter allows linear, quadratic, and cubic models in different regions. Both degree settings use the residual range and maximum-residual subdivision strategies, giving four construction variants. We fix the confidence parameter at $\beta = 10^{-8}$ to obtain a high confidence level. For the violation parameter α, we consider two values, 0.05 and 0.1. We set $\epsilon_{\min} = 25$W for the server or simulator for one GPU and increase it by 5W for each additional active GPU. Fitting, degree selection, and subdivision follow Chapter 5.

We then use the constructed estimators in power control when power meter readings are unreliable. Here P denotes actual server power, $\tilde{P}$ is the power meter reading, and $\hat{P}$ is the power estimate. These correspond to y, $\tilde{y}$, and $\hat{y}$ in Chapter 6. For each GPU configuration, the power set point and safety threshold are

$$P_{\text{set}} = (600 + 150N_{\text{GPU}})\,\text{W}, \qquad P_{\text{safe}} = (600 + 175N_{\text{GPU}})\,\text{W}. \tag{7.1.1}$$

The controller aims to maintain power around P_{set}. A control period is considered unsafe if the actual power exceeds P_{safe}. The control period on the GPU server is one second.

To evaluate control under sensing errors, we inject faults into the feedback power reading while the selected features are assumed to remain reliably measured. In the software experiments, we also add measurement errors to the feature readings used for prediction, which is described in a later paragraph. We adapt the sensor fault models in [40] to represent five types of erroneous power meter readings. The sensing-circuit mechanisms described below illustrate how the corresponding measurement behaviors can arise. Here $U(a, b)$ denotes a uniform distribution over $[a, b]$, and $P_{\min}$ and $P_{\max}$ are the power meter limits.

- **Offset fault.** An external amplifying stage can introduce a fixed bias into a sensor signal, while time-varying sensor noise produces additional fluctuations [40]. In our tests, we represent additive measurement errors using a random-offset model, $\tilde{P} = P \pm P_{\text{offset}}$, where $P_{\text{offset}} \sim U(0, 50N_{\text{GPU}})$ W.

- **Multiplicative fault.** Changes in the resistive components of a sensor circuit can alter its measurement gain, scaling the output signal [40]. In our tests, we represent multiplicative measurement errors by $\tilde{P} = P \pm c_{\text{mul}} \cdot P$, where $c_{\text{mul}} \sim U(0, 1)$.

- **Stuck-at-Min fault.** A short circuit across the output resistor in the referenced sensor circuit can bring its nominal output to the minimum reference level [40]. In our tests, we represent this behavior by a power meter reading fixed at $\tilde{P} = P_{\text{min}}$ during the fault, regardless of changes in actual power.

- **Stuck-at-Max fault.** An open circuit in the output-resistor path of the referenced sensor circuit can bring its nominal output to the maximum supply-voltage level [40]. In our tests, we represent this behavior by a power meter reading fixed at $\tilde{P} = P_{\text{max}}$ during the fault, regardless of changes in actual power.

- **Out-of-Range fault.** A sensor signal exceeding the effective measurement range can cause saturation and clipping. Faults in the sensor supply circuit can also restrict this range [40]. In our tests, an erroneous power reading outside the measurable range is clipped at the corresponding meter limit. A value above P_{max} is returned as P_{max}, while a value below P_{min} is returned as P_{min}.

- **Permanent fault.** The power meter always returns an invalid reading. We indicate this value by 0. We evaluate this case separately because it represents sustained loss of valid power reading.

Except for permanent failure, all errors occur occasionally. For each type, we consider occurrence probabilities of 10% and 50% per control period, referred to as the low and high error rates, respectively. For offset and multiplicative faults, P_{offset} and c_{mul} are sampled in each control period from their specified distributions. Specifically, in the hardware experiments, all erroneous readings $\tilde{P}$ are clipped to the measurable range $[P_{\text{min}}, P_{\text{max}}]$ as a protective measure for the server.

Software Simulation Settings. For both simulators, we conduct the online fault experiments as follows. For each GPU configuration, we test each of the fault types at both occurrence rates. We perform 20 independent executions for each case, selecting an initial state randomly from the safe space for each execution. Each execution lasts 5000 control periods. In each comparison, we start the simulator from the same initial state for all methods and introduce faults in the same control periods.

To model noise in the on-board sensors, we add an independent error between -5% and 5% of the feature value to each reading used for prediction. We add these errors during the online tests, without directly changing the simulated state or the fixed construction datasets.

All software simulation tests are performed on a PC equipped with an AMD Ryzen 9 9900X CPU and 32 GB of DDR5 memory. Only the CPU is used for simulation.

Compared Baselines. For online control, we use $\alpha = 0.05$ and the residual range estimators with $m_{\max} = 1$ and $m_{\max} = 3$. Our framework selects the control mode as described in Section 6.1. We compare it with the following methods in software simulation and hardware testing.

Original Controller. The original controller in [28] operating without power meter errors provides the normal-operation reference. This baseline supplies the feedback power reading $\tilde{P}$ directly to the original controller of each platform. When a fault occurs, the controller continues to use the erroneous reading without additional filtering or fault detection. We use this baseline to assess how the original controller performs under power meter errors when none of the additional methods is applied.

Moving Average (MA). This baseline smooths power meter readings by computing their arithmetic mean over a sliding window [41]. In our experiments, the window covers three control periods, and the averaged reading is supplied to the original controller as feedback. The averaging procedure does not distinguish reliable readings from erroneous ones, so faulty readings also contribute to the feedback. We use this baseline to assess whether simple temporal averaging can support power control under the tested power meter errors.

Kalman Filtering (KF). This baseline recursively estimates server power from the current power meter reading and the previous state estimate [1]. The process noise covariance Q and measurement noise covariance R are estimated from trustworthy data before the fault experiments. Here Q captures the temporal variation of power measurements, while R is computed from the variance of measurements collected under identical operating conditions. At each control period, the resulting power estimate is supplied to the original controller as feedback. The filter continues to incorporate power meter readings when faults occur. We use this baseline to assess whether recursive filtering informed by the power variation and measurement noise in trustworthy data can support power control under the tested power meter errors.

Immediate Conservative Control. This baseline is motivated by the model-based real-time recovery approach in [6], which steers a system toward a target state after a sensor attack is detected. That approach relies on a model of the system dynamics. Since such a model is unavailable for our GPU server, we use our PAC estimators for error detection and the available conservative controller for the response. Once a reading is judged inconsistent, it immediately selects conservative control instead of using predicted feedback. This baseline can therefore be viewed as a more conservative version of our framework, allowing us to assess the effect of prediction-based control after an inconsistency is detected.

N_{GPU}	α	$m_{\max}$	K	$K_{\leq \epsilon_{\min}}$	ϵ_{lo}	ϵ_{hi}	Time(s)	L_{safe}
1	0.1	1						
		3						
	0.05	1						
		3						
2	0.1	1						
		3						
	0.05	1						
		3						
4	0.1	1						
		3						
	0.05	1						
		3						

Table 7.1: Evaluation of PAC estimators on Simulator I. The results from the maximum-residual subdivision are shown in parentheses, while those outside parentheses use residual range subdivision. Legend - K: number of subdivision; $K_{\leq \epsilon_{\min}}$: number of subdivisions $\epsilon \leq \epsilon_{\min}$; $\epsilon_{\text{lo}}/\epsilon_{\text{hi}}$: smallest/largest fitted error bounds among these subdivisions; Time: total construction time of the estimator; L_{safe}: evaluated safe prediction horizon.**Zhuo** fill in the data

7.1.2 Evaluation on Simulator I

Simulator I uses the multivariate linear model of GPU server dynamics developed in [28]. Its MPC controller determines adjustments to CPU and GPU frequencies to regulate server power. For power estimation, the feature vector $\mathbf{x} \in \mathbb{R}^d$ contains one CPU frequency and a separate frequency for each active GPU, giving $d = 1 + N_{\text{GPU}}$. All frequencies are expressed in Mhz.

For each GPU configuration, we collect $N = 20000 N_{\text{GPU}}$ samples from Simulator I following the trustworthy-data collection procedure in Section 7.1.1. These samples form the construction dataset for the power estimators evaluated below.

Performance Evaluation of PAC Estimator. Before using estimated power as feedback, we evaluate the efficiency and accuracy of our data-driven estimator construction method on the given set of trustworthy data. Table 7.2 summarizes the results for Simulator I. For each fixed estimator, we also evaluate L_{safe} with Simulator I's MPC controller using Algorithm 4. The resulting horizon is recorded in Table 7.2 and remains fixed during the corresponding online experiments.

Zhuo analysis required here

Online Power Control. We use the estimators and L_{safe} values obtained above to test power control under the faults described in Section 7.1.1. For each execution, we record the fraction of unsafe control periods. We also calculate the mean absolute difference between actual power and

N_{GPU}	α	$m_{\max}$	K	$K_{\leq \epsilon_{\min}}$	ϵ_{lo}	ϵ_{hi}	Time(s)	L_{safe}
1	0.1	1						
		3						
	0.05	1						
		3						
2	0.1	1						
		3						
	0.05	1						
		3						
4	0.1	1						
		3						
	0.05	1						
		3						

Table 7.2: Evaluation of PAC estimators on Simulator II. The results from the maximum-residual subdivision are shown in parentheses, while those outside parentheses use residual range subdivision. Legend - K: number of subdivision; $K_{\leq \epsilon_{\min}}$: number of subdivisions $\epsilon \leq \epsilon_{\min}$; $\epsilon_{\text{lo}}/\epsilon_{\text{hi}}$: smallest/largest fitted error bounds among these subdivisions; Time: total construction time of the estimator; L_{safe}: evaluated safe prediction horizon.**Zhuo** fill in the data

P_{set}, divide it by P_{set}, and express the result as a percentage. We report the average of each measure and how much it varies between executions. A method can avoid unsafe power levels by keeping power low, so we use both measures to assess its behavior.

We first compare our framework with the original controller, MA, and KF. For each estimator, we then compare our framework with its immediate conservative baseline. We record how often these two methods use the power meter reading, the predicted power, or the conservative controller. Together with the safety and tracking results, this shows whether using predictions reduces conservative control and keeps power closer to the set point.

We also compare the two maximum-degree settings using the same GPU configuration and faults. To explain any differences, we examine the degrees and error bounds of the models selected during execution, together with each estimator's L_{safe}.

For permanent failure, we introduce the fault in the same control period for all methods. We record actual power and the control mode in each period to see how long predictions are used consecutively and how the framework responds if this count reaches L_{safe}.

Simulator I provides a linear reference for the next experiment. We now examine estimator construction and online control for the neural-network power mapping used by Simulator II.

7.1.3 Evaluation on Simulator II

Power Estimator Evaluation.

Online Power Control. The software experiments examine estimator construction and fault handling in a simplified execution environment. We next evaluate the framework on the GPU server, where task loading, hardware execution overheads, and physical protection also affect control and application performance.

7.1.4 Evaluation on the GPU Server

We conduct our experiments on a server that hosts three Nvidia Tesla V100 16 GB GPUs. The server runs Ubuntu 20.04.6 with kernel 5.15.0. It is equipped with an Intel Xeon Gold 5215 @ 2.20 GHz processor (40 cores) and 128 GB of RAM. CapGPU [28] provides the original power controller. A reliable power meter, called the monitor, records actual server power. The feedback reading supplied to each evaluated method is affected by the injected fault. The monitor remains separate from this feedback path, except for the physical protection described below.

For each configuration, the nominal construction dataset contains $N = 20000N_{\mathrm{GPU}}$ samples.

Power Estimator Evaluation.

Online Power Control.

Application Performance.

Online Overhead.

Chapter 8

Conclusion

Bibliography

[1] R. E. Kalman, "A new approach to linear filtering and prediction problems," 1960.

[2] M. Pajic, J. Weimer, N. Bezzo, O. Sokolsky, G. J. Pappas, and I. Lee, "Design and implementation of attack-resilient cyberphysical systems: With a focus on attack-resilient state estimators," *IEEE Control Systems Magazine*, vol. 37, no. 2, pp. 66–81, 2017.

[3] R. Ivanov, M. Pajic, and I. Lee, "Attack-resilient sensor fusion for safety-critical cyber-physical systems," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 15, no. 1, p. 21, 2016.

[4] H. Choi, W.-C. Lee, Y. Aafer, F. Fei, Z. Tu, X. Zhang, D. Xu, and X. Deng, "Detecting attacks against robotic vehicles: A control invariant approach," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 801–816.

[5] R. Quinonez, J. Giraldo, L. Salazar, E. Bauman, A. Cardenas, and Z. Lin, "SAVIOR: Securing autonomous vehicles with robust physical invariants," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, Aug. 2020, pp. 895–912.

[6] L. Zhang, X. Chen, F. Kong, and A. A. Cardenas, "Real-time attack-recovery for cyber-physical systems using linear approximations," in *2020 IEEE Real-Time Systems Symposium (RTSS)*, 2020, pp. 205–217.

[7] P. Kadlec, B. Gabrys, and S. Strandt, "Data-driven soft sensors in the process industry," *Computers Chemical Engineering*, vol. 33, no. 4, pp. 795–814, 2009.

[8] L. Zhang, K. Sridhar, M. Liu, P. Lu, X. Chen, F. Kong, O. Sokolsky, and I. Lee, "Real-time data-predictive attack-recovery for complex cyber-physical systems," in *2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2023, pp. 209–222.

[9] D. Haussler, "Probably approximately correct learning," in *Proceedings of the Eighth National Conference on Artificial Intelligence - Volume 2*, ser. AAAI'90. AAAI Press, 1990, p. 1101–1108.

[10] G. Calafiore and M. Campi, "The scenario approach to robust control design," *IEEE Transactions on Automatic Control*, vol. 51, no. 5, pp. 742–753, 2006.

[11] M. C. Campi, S. Garatti, and M. Prandini, "The scenario approach for systems and control design," *IFAC Proceedings Volumes*, vol. 41, no. 2, pp. 381–389, 2008, 17th IFAC World Congress.

[12] B. Xue, M. Zhang, A. Easwaran, and Q. Li, "Pac model checking of black-box continuous-time dynamical systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3944–3955, 2020.

[13] R. Li, P. Yang, C.-C. Huang, Y. Sun, B. Xue, and L. Zhang, "Towards practical robustness analysis for dnns based on pac-model learning," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2189–2201.

[14] C. Fan, B. Qi, S. Mitra, and M. Viswanathan, "Dryvr: Data-driven verification and compositional reasoning for automotive systems," in *Computer Aided Verification*, R. Majumdar and V. Kunčak, Eds. Cham: Springer International Publishing, 2017, pp. 441–461.

[15] D. Sun and S. Mitra, "Neureach: Learning reachability functions from simulations," in *Tools and Algorithms for the Construction and Analysis of Systems*, D. Fisman and G. Rosu, Eds. Cham: Springer International Publishing, 2022, pp. 322–337.

[16] E. Dietrich, A. Devonport, and M. Arcak, "Nonconvex scenario optimization for data-driven reachability," in *Proc. of L4DC'24*, ser. Proceedings of Machine Learning Research, vol. 242. PMLR, 2024, pp. 514–527.

[17] K. Polymenakos, L. Laurenti, A. Patane, J.-P. Calliess, L. Cardelli, M. Kwiatkowska, A. Abate, and S. Roberts, "Safety guarantees for iterative predictions with gaussian processes," in *Proc. of CDC'20*, 2020, pp. 3187–3193.

[18] Y.-F. Chen, C. Hsieh, O. Lengál, T.-J. Lii, M.-H. Tsai, B.-Y. Wang, and F. Wang, "Pac learning-based verification and model synthesis," in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 714–724.

[19] P. Ashok, J. Křetínský, and M. Weininger, "Pac statistical model checking for markov decision processes and stochastic games," in *Computer Aided Verification*, I. Dillig and S. Tasiran, Eds. Cham: Springer International Publishing, 2019, pp. 497–519.

[20] S. K. Jha, E. M. Clarke, C. J. Langmead, A. Legay, A. Platzer, and P. Zuliani, "A bayesian approach to model checking biological systems," in *Computational Methods in Systems Biology*, P. Degano and R. Gorrieri, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 218–234.

[21] M. Liu, P. Lu, X. Chen, O. Sokolsky, I. Lee, and F. Kong, "Deadline-safe reach-avoid control synthesis for cyber-physical systems with reinforcement learning," in *2024 IEEE Real-Time Systems Symposium (RTSS)*, 2024, pp. 96–108.

[22] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The simplex architecture for safe online control system upgrades," in *Proceedings of the 1998 American Control Conference. ACC (IEEE Cat. No.98CH36207)*, vol. 6, 1998, pp. 3504–3508 vol.6.

[23] S. Bak, A. Greer, and S. Mitra, "Hybrid cyberphysical system verification with simplex using discrete abstractions," in *Proc. of RTAS'10*. IEEE Computer Society, 2010, pp. 143–152.

[24] S. Bak, T. T. Johnson, M. Caccamo, and L. Sha, "Real-time reachability for verified simplex design," in *2014 IEEE Real-Time Systems Symposium*, 2014, pp. 138–148.

[25] C. Lefurgy, X. Wang, and M. Ware, "Server-level power control," in *Fourth International Conference on Autonomic Computing (ICAC'07)*, 2007, pp. 4–4.

[26] V. Sakalkar, V. Kontorinis, D. Landhuis, S. Li, D. De Ronde, T. Blooming, A. Ramesh, J. Kennedy, C. Malone, J. Clidaras, and P. Ranganathan, "Data center power oversubscription with a medium voltage power plane and priority-aware capping," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 497–511.

[27] A. Kumbhare, R. Azimi, I. Manousakis, A. Bonde, F. Frujeri, N. Mahalingam, P. Misra, S. A. Javadi, B. Schroeder, M. Fontoura, and R. Bianchini, "Prediction-based power oversubscription in cloud platforms," 2020.

[28] Y. Ma, S. Subramaniyan, and X. Wang, "Power capping of gpu servers for machine learning inference optimization," in *Proceedings of the 54th International Conference on Parallel Processing*, ser. ICPP '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 449–459.

[29] S. Hong and H. Kim, "An integrated gpu power and performance model," *SIGARCH Comput. Archit. News*, vol. 38, no. 3, p. 280–289, Jun. 2010.

[30] R. A. Bridges, N. Imam, and T. M. Mintz, "Understanding gpu power: A survey of profiling, modeling, and simulation methods," *ACM Comput. Surv.*, vol. 49, no. 3, Sep. 2016.

[31] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "Accelwattch: A power modeling framework for modern gpus," in *Proc. of MICRO'21*. ACM, 2021, p. 738–753.

[32] H. Carstens, X. Xia, and S. Yadavalli, "Measurement uncertainty in energy monitoring: Present state of the art," *Renewable and Sustainable Energy Reviews*, vol. 82, pp. 2791–2805, 2018.

[33] D. Masti, D. Bernardini, and A. Bemporad, "A machine-learning approach to synthesize virtual sensors for parameter-varying systems," *European Journal of Control*, vol. 61, pp. 40–49, 2021.

[34] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.

[35] P. A. Parrilo, "Semidefinite programming relaxations for semialgebraic problems," *Mathematical programming*, vol. 96, no. 2, pp. 293–320, 2003.

[36] X. Zeng, B. Liu, Z. Zeng, Z. Liu, and Z. Yang, "Safe controller synthesis for nonlinear systems via reinforcement learning and pac approximation," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024.

[37] T. M. Mitchell and T. M. Mitchell, *Machine learning*. McGraw-hill New York, 1997, vol. 1, no. 9.

[38] Y. Zhang, S. Sankaranarayanan, and F. Somenzi, "Statistically sound verification and optimization for complex systems," in *Automated Technology for Verification and Analysis*, F. Cassez and J.-F. Raskin, Eds. Cham: Springer International Publishing, 2014, pp. 411–427.

[39] R. Tempo, E. Bai, and F. Dabbene, "Probabilistic robustness analysis: Explicit bounds for the minimum number of samples," *Systems Control Letters*, vol. 30, no. 5, pp. 237–242, 1997.

[40] F. Mehmood, P. M. Papadopoulos, L. Hadjidemetriou, and M. M. Polycarpou, "Modeling of sensor faults in power electronics inverters and impact assessment on power quality," in *2021 IEEE Madrid PowerTech*, 2021, pp. 1–6.

[41] S. W. Smith, *The scientist and engineer's guide to digital signal processing*. USA: California Technical Publishing, 1997.